Les types de variables

Démonstration 02 du module 4

L'objectif de cette démonstration est de découvrir comment nous pouvons lier les variables entre elles, en gardant à l'esprit que les variables doivent être de type identique.

Déroulement

- Dans le cas où nous disposons de deux variables de même type, la variable contenant le résultat sera du même type que les variables d'origine.
- Voici ce qu'il est possible de faire avec des variables de type `String`

```
PS C:\> $a = "abc"
PS C:\> $b = "def"
PS C:\> $c = $a + $b
PS C:\> $c
abcdef
```

Les variables de type `String` peuvent être fusionnées pour obtenir une chaîne de caractères.

```
PS C:\> $d = Get-ADUser -filter *
PS C:\> $e = $d.name
PS C:\> $f = $d.givenname
PS C:\> $g = $e + $f
PS C:\> $g
Administrator
Guest
krbtgt
toto
System.Collections.Hashtable
Morse
Yang
Randall
```

On récupère les valeurs des propriétés avec le « . » puis on le stocke en variable.

```
PS C:\> $g.gettype().fullname
System.String
```

La variable finale conserve le type `SystemString` et hérite des variables `$e` et `$f`.

Le type de variable `int` va interpréter les chiffres et va permettre d'utiliser les opérateurs mathématiques.

© ENI – Tous droits réservés

```
PS C:\> $a = 1
PS C:\> $b = 2
PS C:\> $c = $a + $b
PS C:\> $c
3
```

Il est possible de récupérer une valeur de type `Int` sur un objet.

```
PS C:\> $d = Get-Process
PS C:\> $e = $d[4].id
PS C:\> $f = $d[5].id
PS C:\> $g = $e / $f
PS C:\> $g
0.0936026936026936
```

```
PS C:\>  [math]::Round($g,2)
0.09
```

Cette méthode appliquée à la variable `$g` arrondit le nombre de décimales à la valeur indiquée entre les parenthèses.

Le troisième type de variable est le type `Array` qui permet de stocker des valeurs en leur attribuant un index.

```
PS C:\> $array = "pc1","pc2","pc3"
PS C:\> $array[1]
pc2
PS C:\> $array.GetType().fullname
System.Object[]
```

La variable `$array` est de type `System.Object` car les variables de type `Array` sont utilisées pour stocker des objets sur des résultats de commande comme `Get-Aduser`.

L'avantage de ces variables est qu'il est possible de leur appliquer des méthodes (puisqu'étant de type `System.Object`)

```
PS C:\> $array.Replace("PC1","PC11")
pc1
pc2
pc3
PC4
PS C:\> $array.Replace("pc1","PC11")
PC11
pc2
pc3
PC4
```

Attention la méthode `Replace` est sensible à la casse sur les valeurs entre les parenthèses.

```
PS C:\> $array[3]
PC4
```


© ENI – Tous droits réservés

Attention également à l'indexation qui débute à 0 ; ainsi la valeur d'index 3 pointe vers la quatrième valeur du tableau.

Toutefois la variable de type `array` est statique ; le nombre de valeurs ne peut être diminué.

© ENI – Tous droits réservés